Photobooth CRM Developer Handover

Setup, deployment, environment configuration, Stripe integration, and payment workflow documentation.

Project: Photobooth CRM / MemoShot multi-tenant Laravel application

Prepared: 21 April 2026

1. Application Overview

This application is a Laravel and Vue multi-tenant CRM for photobooth businesses. Each tenant is represented by a workspace subdomain, for example `acme.memoshot.com`. Tenants manage bookings, invoices, packages, equipment, add-ons, discounts, leads, customers, campaigns, templates, subscriber groups, workspace settings, and tenant-owned Stripe payment settings.

The central domain also contains a super admin area at `/admin`. The super admin manages tenants, subscription plans, tenant access, platform subscription history, unpaid subscription charges, server utility commands, and the production `.env` editor.

Important Separation

Area	Purpose	Stripe Account Used	Money Goes To
Tenant invoice payments	Customer pays an invoice/installment for a booking.	Tenant Stripe secret from Workspace Settings.	Tenant/business owner.
Platform subscription payments	Tenant pays MemoShot/platform for CRM subscription.	Platform Stripe secret from `.env`.	Platform owner.

2. Technology Stack

- PHP/Laravel 13.x application.
- Vue admin frontend compiled by Vite.
- MySQL or SQLite depending on environment.
- Mailpit/Mailhog for local mail testing; SMTP/log mailer for other environments.
- Stripe Checkout and Payment Intents for payments.
- Dompdf is installed for PDF generation inside the app.

3. Local Setup

The repository is expected at:

```
/Users/junedelacruz/Desktop/aaa-project/photobooth-crm
```

Install Dependencies

```
composer install
npm install
```

Create Local Environment

```
cp .env.example .env
php artisan key:generate
```

For local SQLite development, use:

```
DB_CONNECTION=sqlite
DB_DATABASE=database/database.sqlite
```

Create the SQLite file if it does not exist:

```
mkdir -p database
touch database/database.sqlite
```

Run Migrations and Seeders

```
php artisan migrate --force
php artisan db:seed --class=MemoShotSeeder --force
php artisan db:seed --class=DemoSeeder --force
```

Storage Symlink

```
php artisan storage:link
```

Build Frontend Assets

```
npm run build
```

Docker Local Stack

The project is also configured for the shared local Docker stack at:

```
/Users/junedelacruz/Desktop/aaa-project/docker/compose.yaml
```

Common Makefile commands:

```
make shell
make artisan migrate --force
make deploy-build
```

4. Important URLs

URL	Purpose
`https://memoshot.com`	Central production domain.
`https://acme.memoshot.com`	Example tenant workspace domain.
`https://memoshot.com/admin`	Super admin area.
`https://memoshot.com/stripe/webhook`	Tenant/customer invoice payment webhook endpoint.
`https://memoshot.com/platform/stripe/webhook`	Platform subscription payment webhook endpoint.

5. Environment Variables

Never commit a real production `.env`. Use `.env.example` as the baseline and fill production values on the server.

Core App

Key	Purpose	Production Example
`APP_NAME`	Displayed application name.	`MemoShot`
`APP_ENV`	Laravel environment.	`production`
`APP_DEBUG`	Disable detailed errors in production.	`false`
`APP_URL`	Base central app URL.	`https://memoshot.com`
`TENANT_BASE_DOMAIN`	Base domain used to resolve tenant subdomains.	`memoshot.com`
`SUPER_ADMIN`	Email allowed to access super admin login.	`jtpayoran@yahoo.com`

Database

Key	Purpose
`DB_CONNECTION`	`sqlite` locally or `mysql` on shared hosting/production.

Key	Purpose
`DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`	Required for MySQL production deployments.

Mail

Key	Purpose
`MAIL_MAILER`, `MAIL_HOST`, `MAIL_PORT`	Mailer driver and host. Local Docker uses Mailpit/Mailhog. Production should use a real SMTP provider.
`MAIL_USERNAME`, `MAIL_PASSWORD`	SMTP credentials where required.
`MAIL_FROM_ADDRESS`, `MAIL_FROM_NAME`	Default sender for login codes, invoices, campaigns, and platform billing emails.

Google Maps

Google Maps is app-level, not tenant-level.

```
VITE_GOOGLE_MAPS_API_KEY=your_google_maps_javascript_api_key
```

Enable the needed Google Maps services in Google Cloud, especially Maps JavaScript API, Places API, and Geocoding API when address suggestions or distance calculations are required.

Platform Stripe Environment

```
PLATFORM_STRIPE_SECRET=sk_live_or_test_key
PLATFORM_STRIPE_WEBHOOK_SECRET=whsec_platform_webhook_secret
PLATFORM_STRIPE_CURRENCY=usd
PLATFORM_SUBSCRIPTION_AUTO_CHARGE=true
PLATFORM_SUBSCRIPTION_REMINDER_DAYS=5
PLATFORM_SUBSCRIPTION_RETRY_DAYS=5
PLATFORM_SUBSCRIPTION_SCHEDULE_TIME=09:00
PLATFORM_SUBSCRIPTION_ADMIN_EMAIL=admin@example.com
```

Key	Purpose
`PLATFORM_STRIPE_SECRET`	Creates platform subscription Checkout sessions and off-session charges.
`PLATFORM_STRIPE_WEBHOOK_SECRET`	Verifies Stripe webhook signatures for `/platform/stripe/webhook`.
`PLATFORM_SUBSCRIPTION_AUTO_CHARGE`	Turns automatic platform subscription deduction on or off.
`PLATFORM_SUBSCRIPTION_REMINDER_DAYS`	Number of days before due date to email the tenant about the upcoming charge.

Key	Purpose
`PLATFORM_SUBSCRIPTION_RETRY_DAYS`	Number of days before retrying a failed platform subscription payment.
`PLATFORM_SUBSCRIPTION_SCHEDULE_TIME`	Daily scheduler time for subscription processing.

Do not use tenant Stripe keys for platform subscription payment, and do not use platform Stripe keys for tenant invoice payments.

6. Tenant Stripe Settings

Each tenant configures its own Stripe values under **Settings > Workspace Settings**. These tenant values are used only for customer invoice/installment payments.

Workspace Setting	Purpose
Stripe Secret Key	Tenant-owned Stripe secret key. Used to create Checkout sessions for customer invoice payments.
Tenant Stripe Webhook URL	Read-only helper showing `https://memoshot.com/stripe/webhook`.
Stripe Webhook Secret	Tenant-owned webhook signing secret from that tenant's Stripe Dashboard webhook endpoint.
Stripe Currency	Three-letter currency for tenant checkout sessions, for example `aud`.

7. Stripe Setup

Tenant Customer Invoice Payments

1. Tenant logs in to their own Stripe Dashboard.
2. Create a webhook endpoint with URL `https://memoshot.com/stripe/webhook`.
3. Select event `checkout.session.completed`.
4. Copy the generated `whsec_...` signing secret.
5. Paste it into Workspace Settings as the tenant Stripe Webhook Secret.
6. Paste the tenant Stripe Secret Key into Workspace Settings.
7. Set the tenant Stripe Currency.

Tenant invoice payments include `invoice_id` and `installment_id` metadata. The webhook verifies the tenant-specific webhook secret before marking the installment as paid.

Platform Subscription Payments

1. Platform owner logs in to the platform Stripe account.
2. Create a webhook endpoint with URL `https://memoshot.com/platform/stripe/webhook`.
3. Select event `checkout.session.completed`.
4. Copy the generated `whsec_...` signing secret.
5. Set it in production `.env` as `PLATFORM_STRIPE_WEBHOOK_SECRET`.
6. Set `PLATFORM_STRIPE_SECRET` to the platform Stripe secret key.

Platform subscription payments include `tenant_subscription_charge_id` metadata. They also save Stripe customer/payment method IDs for future off-session subscription billing.

Webhook Endpoints

Endpoint	Stripe Source	Event	Secret Location
`/stripe/webhook`	Tenant Stripe account	`checkout.session.completed`	Workspace Settings
`/platform/stripe/webhook`	Platform Stripe account	`checkout.session.completed`	Production `.env`

8. Payment Workflows

Customer Invoice Payment Flow

1. Admin creates or sends an invoice for a booking.
2. Customer opens the invoice payment link or pays from the public invoice page.
3. Application creates a Stripe Checkout session using the tenant's Stripe secret.
4. Customer pays in Stripe Checkout.
5. Stripe sends `checkout.session.completed` to `/stripe/webhook`.
6. App verifies the tenant webhook secret, marks the installment paid, recalculates invoice amount paid/status, and updates booking status.
7. As a fallback, the success return URL includes `session_id`, and the app verifies the Checkout session directly with the tenant's Stripe secret before marking paid.

Platform Subscription Payment Flow

1. Super admin creates subscription plans.
2. Tenant selects a monthly, quarterly, yearly, or weekly plan in Workspace Settings.
3. Tenant clicks pay subscription.
4. Application creates a platform Stripe Checkout session using `PLATFORM_STRIPE_SECRET`.
5. Customer/payment method is saved in Stripe for future off-session billing. The app saves only Stripe IDs such as `cus_...` and `pm_...`.
6. Webhook or verified return URL marks the tenant subscription charge as paid.
7. Scheduled billing creates future charges, sends reminders, charges due subscriptions, sends thank-you emails, and retries failures.

The application does not store card numbers, expiry dates, or CVC values. Those remain in Stripe.

Platform Subscription Emails

- Reminder email: sent before the due date based on `PLATFORM_SUBSCRIPTION_REMINDER_DAYS`.

- Paid email: sent after successful platform subscription payment.
- Failed email: sent after a failed automatic deduction, with retry timing based on `PLATFORM_SUBSCRIPTION_RETRY_DAYS`.

9. Scheduled Billing

The scheduler runs the platform subscription billing command daily at the configured time:

```
php artisan platform-subscriptions:process
```

Manual production command:

```
/opt/alt/php84/usr/bin/php artisan platform-subscriptions:process
```

Hostinger cron should run Laravel scheduler once per minute:

```
cd /home/u838520432/domains/memoshot.com/laravel_app && /opt/alt/php84/usr/bin/php artisan schedule:run >>
/dev/null 2>&1
```

10. Deployment on Hostinger Shared Hosting

The app uses a sibling folder layout:

```
/home/u838520432/domains/memoshot.com/laravel_app
/home/u838520432/domains/memoshot.com/public_html
```

The Laravel app should live outside `public_html`. The generated `public_html/index.php` points to the Laravel app path.

SSH

```
ssh -p 65002 u838520432@157.173.209.64
```

Deployment Make Commands

Command	Purpose
`make deploy-build`	Install production PHP dependencies, install Node dependencies, build frontend, clear optimize cache.
`make deploy-ssh`	Rsync Laravel app files to remote app folder without deleting removed remote files.
`make deploy-ssh-public`	Generate and upload public web root files to `public_html`.
`make deploy-ssh-all`	Run app and public rsync upload.
`make deploy-ssh-delete`	Rsync app with `--delete`. Use carefully.

Deployment Environment File

`.env.deploy` is not committed. It should contain SSH deployment settings similar to:

```
SSH_HOST=157.173.209.64
SSH_PORT=65002
SSH_USER=u838520432
SSH_REMOTE_APP_DIR=/home/u838520432/domains/memoshot.com/laravel_app
SSH_REMOTE_PUBLIC_DIR=/home/u838520432/domains/memoshot.com/public_html
DEPLOY_SERVER_APP_PATH=/home/u838520432/domains/memoshot.com/laravel_app
```

Post-Deploy Server Commands

```
cd /home/u838520432/domains/memoshot.com/laravel_app
/opt/alt/php84/usr/bin/php artisan migrate --force
```

```
/opt/alt/php84/usr/bin/php artisan optimize:clear
/opt/alt/php84/usr/bin/php artisan config:cache
/opt/alt/php84/usr/bin/php artisan route:cache
/opt/alt/php84/usr/bin/php artisan view:cache
```

Storage Symlink on Hostinger

```
cd /home/u838520432/domains/memoshot.com
ln -sfn laravel_app/storage/app/public public_html/storage
```

11. Tenant Domains and Wildcard DNS

Tenants are resolved by subdomain. For dynamic tenants such as `acme.memoshot.com`, configure Hostinger/DNS to support wildcard subdomains.

- Create wildcard DNS record: `*.memoshot.com` pointing to the hosting IP.
- Ensure the web server routes wildcard subdomains to the same Laravel app.
- Ensure SSL covers wildcard subdomains or individual tenant subdomains as required by the hosting provider.

12. Super Admin

Super admin login uses email code verification. Configure:

```
SUPER_ADMIN=jtpayoran@yahoo.com
```

Super admin capabilities include:

- Maintain subscription plans and validity.
- Assign free-for-life plans that tenants cannot select themselves.
- View tenant-selected subscriptions and payment history.
- Manually enable or disable tenant access.
- View unpaid tenant subscription charges.
- Copy server Artisan commands and Stripe endpoint URLs.
- Edit production `.env` with timestamped backup.

13. Common Artisan Commands

```
/opt/alt/php84/usr/bin/php artisan migrate --force
/opt/alt/php84/usr/bin/php artisan db:seed --class=MemoShotSeeder --force
/opt/alt/php84/usr/bin/php artisan db:seed --class=DemoSeeder --force
/opt/alt/php84/usr/bin/php artisan optimize:clear
/opt/alt/php84/usr/bin/php artisan storage:link
```

```
/opt/alt/php84/usr/bin/php artisan config:cache
/opt/alt/php84/usr/bin/php artisan route:cache
/opt/alt/php84/usr/bin/php artisan view:cache
/opt/alt/php84/usr/bin/php artisan platform-subscriptions:process
```

14. Key Files for New Developers

File	Purpose
`routes/web.php`	Main web routes, admin routes, Stripe webhook routes.
`routes/console.php`	Platform subscription billing command and schedule.
`app/Http/Controllers/StripeWebhookController.php`	Handles tenant and platform Stripe webhooks.
`app/Support/StripeCheckoutLinkGenerator.php`	Creates tenant/customer invoice Checkout links.
`app/Support/PlatformSubscriptionCheckoutLinkGenerator.php`	Creates platform subscription Checkout links.
`app/Services/PlatformSubscriptionBillingService.php`	Creates platform charges, reminders, off-session payments, and retry workflow.
`app/Http/Controllers/SettingsController.php`	Tenant workspace settings and tenant subscription payment entry point.
`app/Http/Controllers/SuperAdminController.php`	Super admin dashboard, plans, tenant access, and .env editor.
`resources/js/admin/pages/SettingsPage.vue`	Tenant workspace settings UI including tenant Stripe values.
`resources/views/super-admin/index.blade.php`	Super admin UI.

15. Troubleshooting

Problem	Likely Fix
Payment succeeds in Stripe but remains pending.	Verify correct webhook endpoint, webhook secret, selected event, and run `php artisan optimize:clear` after changing `.env`.
Tenant invoice payment goes to wrong account.	Confirm tenant Workspace Settings has its own Stripe secret. There should be no global tenant Stripe fallback.

Problem	Likely Fix
Platform subscription auto-charge does not run.	Confirm Hostinger cron calls `schedule:run` every minute and `PLATFORM_SUBSCRIPTION_AUTO_CHARGE=true`.
Automatic charge fails because no payment method exists.	Tenant must complete one platform Checkout payment first so Stripe customer/payment method IDs are saved.
CSS/JS loads from localhost or Vite in production.	Run `npm run build`, deploy `public/build`, and remove any stale `public/hot`.
Please provide a valid cache path.	Ensure `storage/framework/cache`, `storage/framework/views`, and related storage directories exist and are writable.